

שמות הסטודנטים	ת.ז.
דניאל ג'נודי	318852571
הראל מזרחי	322809922
קוד פרויקט	051
מנחה	ד"ר אריאל רוט

Abstract

This project implements SeekSuit, an AI-powered inventory and discovery platform tailored for Janudi Fashion. The system integrates a hybrid visual-textual search engine, an automated mobile cataloging tool, and an autonomous agent that analyzes search trends for business insights. The solution enhances operational efficiency, product discovery, and data-driven business strategy.

1. Introduction

The men's fashion retail domain includes a wide variety of styles, cuts, colors, and seasonal collections, which makes product discovery and inventory utilization challenging, especially for businesses that hold large physical stock without a complete digital catalog. SeekSuit is a web-based platform built for Janudi Fashion to bridge this gap by enabling customers and store managers to access and leverage a large-scale suit inventory through advanced AI capabilities.

2. Problem Statement

Managing a vast physical inventory while transitioning from wholesale to retail creates a major operational bottleneck because producing a high-quality digital catalog typically requires professional studio photography. This process is prohibitively expensive and slow at the required scale. At the same time, the sheer volume and diversity of items makes it difficult for customers to browse efficiently and locate specific styles. This leads to poor product discovery and missed sales opportunities. In parallel, the business lacks real-time visibility into evolving customer demand such as emerging preferences for particular colors or cuts which makes inventory and marketing decisions largely reactive. This project addresses these challenges by introducing a cost-effective mobile digitization workflow with automated image enhancement alongside an AI powered multimodal search using images and natural language. Additionally, an autonomous insight agent continuously analyzes user behavior to generate actionable business recommendations.

3. Objectives

- Develop a Hybrid Search Engine: Implement a dual-mode search system enabling users to locate products via visual similarity and semantic natural language queries.*
- Streamline Catalog Digitization: Implement an automated image enhancement feature that enables the store owner to upload raw images, including simple phone photographs. The system will process these inputs, like performing background removal, lighting correction and centering, to instantly generate studio-quality catalog listings.*
- Deploy an Autonomous Insight Agent: Integrate an LLM-based agent to analyze aggregated trends from both visual uploads and textual queries, providing strategic business recommendations to the store owner.*

4. *Architect a Full-Stack Solution: Build a robust client-side interface for inventory display and search features, backed by a server-side system that manages the autonomous agent, executes the automated image enhancement pipeline, and maintains the large-scale image repository.*

4. User Stories

1. *As a customer, I want to upload a reference photo from any source, so I can find visually similar suits in the store's catalog without knowing their exact names.*
2. *As a customer, I want to search for suits using free-text natural language descriptions, so I can find relevant items based on context rather than specific keywords.*
3. *As a customer, I want to see recommendations for visually similar items when viewing a specific product, so I can explore alternatives if the current one isn't a perfect match.*
4. *As a customer, I want to leave my contact details via a request form, so a store representative can get back to me for personal inquiries.*
5. *As a customer, I want to filter products by standard attributes like size and color, so I can narrow down the catalog using traditional search methods.*
6. *As a customer, I want to view basic store information like location and opening hours, so I can plan my visit or contact the business easily.*
7. *As a store owner, I want to upload raw photos of suits taken with my smartphone, so I can automatically generate studio-quality listings with background removal, lighting correction, and automatic centering.*
8. *As a store owner, I want to receive proactive alerts from the autonomous agent regarding search trends and popular keywords, so I can identify emerging demand regardless of my current stock.*
9. *As a store owner, I want to review and edit the auto-generated tags for new uploaded items, so I can ensure the catalog data is 100% accurate before publishing.*
10. *As a store owner, I want to manually prioritize specific items in search results, so I can promote new collections or help clear out older stock.*
11. *As a store owner, I want to securely log in to the admin dashboard, so I can manage the inventory and view sensitive business insights without unauthorized access.*
12. *As a system administrator, I want to configure the execution frequency of the autonomous insight agent, so I can balance between real-time data analysis and server load.*

Sample Test Code for User Stories

User Story 1: Visual Search

```
# Testing User Story #1: Visual Search
def test_visual_search_accuracy(self):
    """
    Scenario: A user uploads a photo of a black tuxedo.
    Expected: The system returns items tagged as 'black' and 'tuxedo'.
    """

    reference_image = "test_images/user_upload_black_tux.jpg"
    results = self.search_engine.search_by_image(reference_image)

    self.assertTrue(len(results) > 0, "Search should return at least one result")
    top_result = results[0]
    self.assertIn("black", top_result.color)
    self.assertIn("tuxedo", top_result.category)
```

User Story 2: Natural Language Search

```
# Testing User Story #2: Natural Language Search
def test_natural_language_query(self):
    """
    Scenario: User types a complex query 'light blue suit with two buttons'.
    Expected: Each result must match at least one criterion (color='blue' OR buttons=2).
    """

    query = "light blue suit with two buttons"
    results = self.search_engine.search_by_text(query)

    self.assertTrue(len(results) > 0, "Should return results")

    for item in results:
        # Check if item matches at least one of the requested attributes
        matches_color = (item.color == "light blue")
        matches_buttons = (item.button_count == 2)

        self.assertTrue(matches_color or matches_buttons,
            f"Item {item.id} is irrelevant: neither light blue nor has 2 buttons")
```

User Story 8: Agent Trend Analysis

```
# Testing User Story #8: Agent Trend Analysis
def test_agent_trend_alert(self):
    """
    Scenario: Simulation of 500 user searches for 'Velvet Jackets' over 24 hours.
    Expected: Agent generates a 'High Demand' alert for 'Velvet'.
    """

    # Simulate search history
    search_log = ["velvet jacket"] * 500
    self.agent.ingest_data(search_log)

    alerts = self.agent.generate_daily_report()

    self.assertTrue(any(a.keyword == "velvet" for a in alerts))
```

User Story 10: Manual Item Prioritization

```
# Testing User Story #10: Manual Item Prioritization
def test_manual_prioritization(self):
    """
    Scenario: Store owner manually boosts the priority of a specific suit (e.g., 'New Arrival').
    Expected: This suit appears at the top of search results, overriding default sorting.
    """

    # Create two items: one standard, one promoted
    standard_item = Product(id=1, name="Standard Grey Suit", priority=0)
    promoted_item = Product(id=2, name="New Collection Black Suit", priority=10)

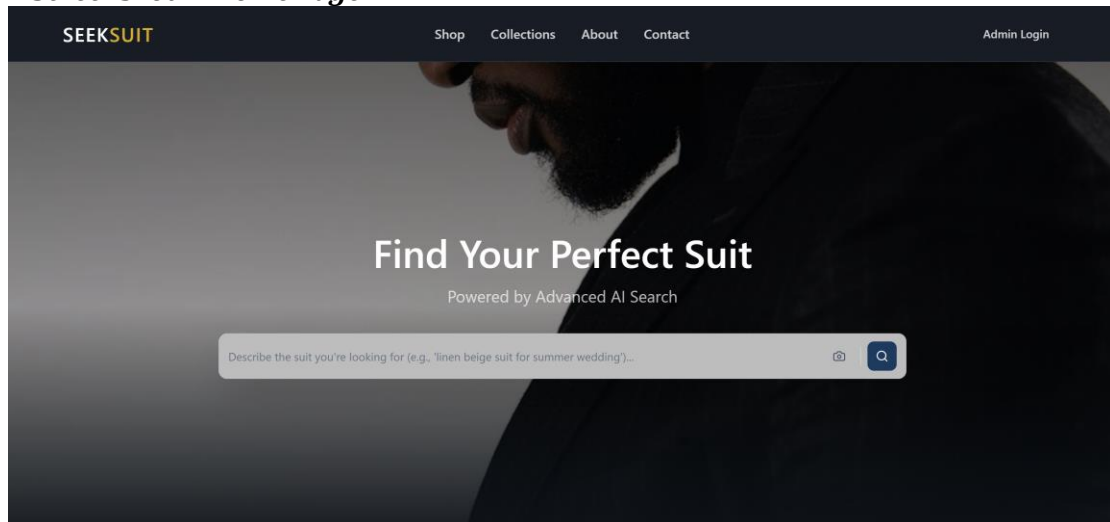
    self.inventory.add([standard_item, promoted_item])

    # Perform a generic search that matches both
    results = self.search_engine.get_all_items()

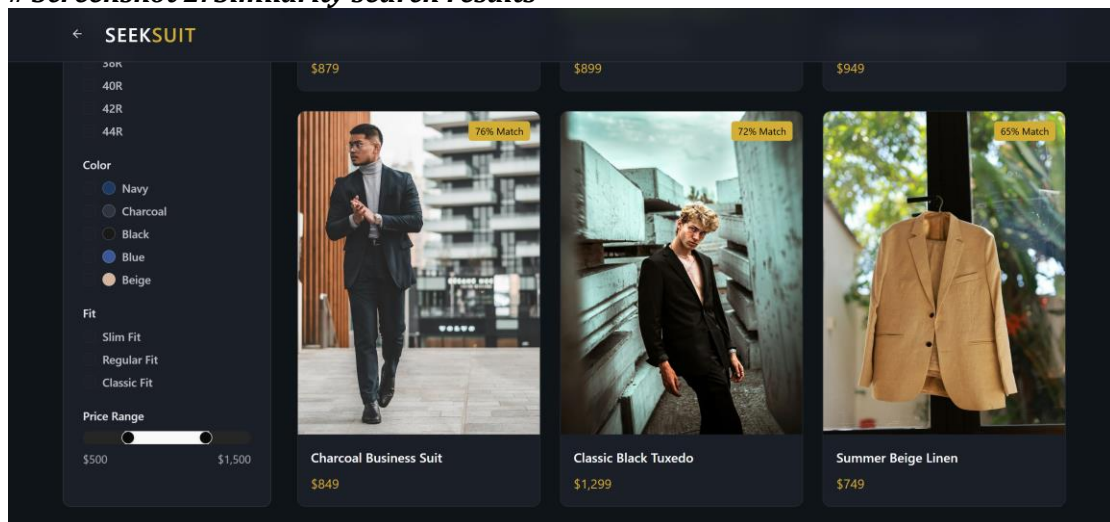
    # Verify the promoted item is first
    self.assertEqual(results[0].id, promoted_item.id)
    self.assertEqual(results[0].name, "New Collection Black Suit")
```

5. Application Screenshots

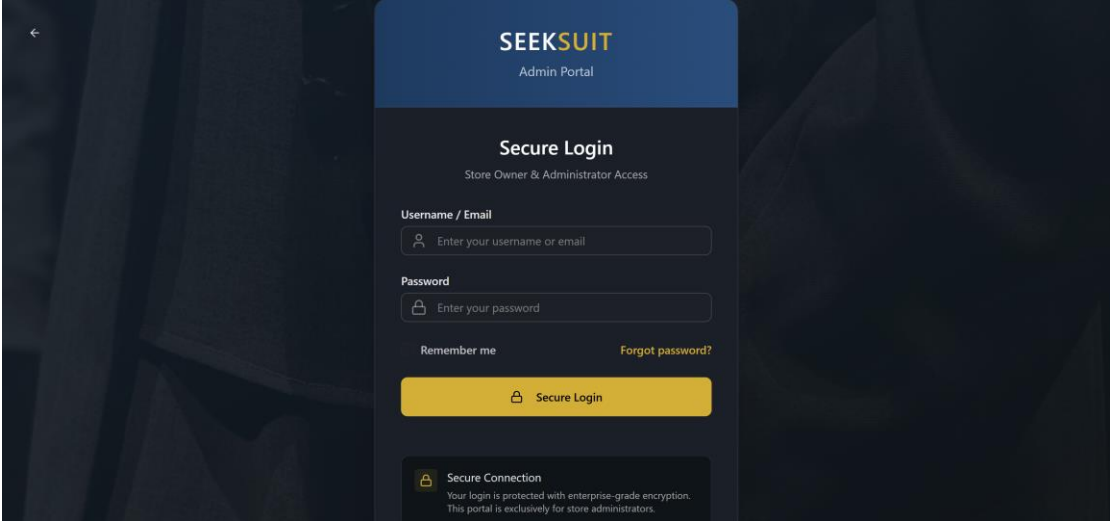
Screenshot 1: Home Page



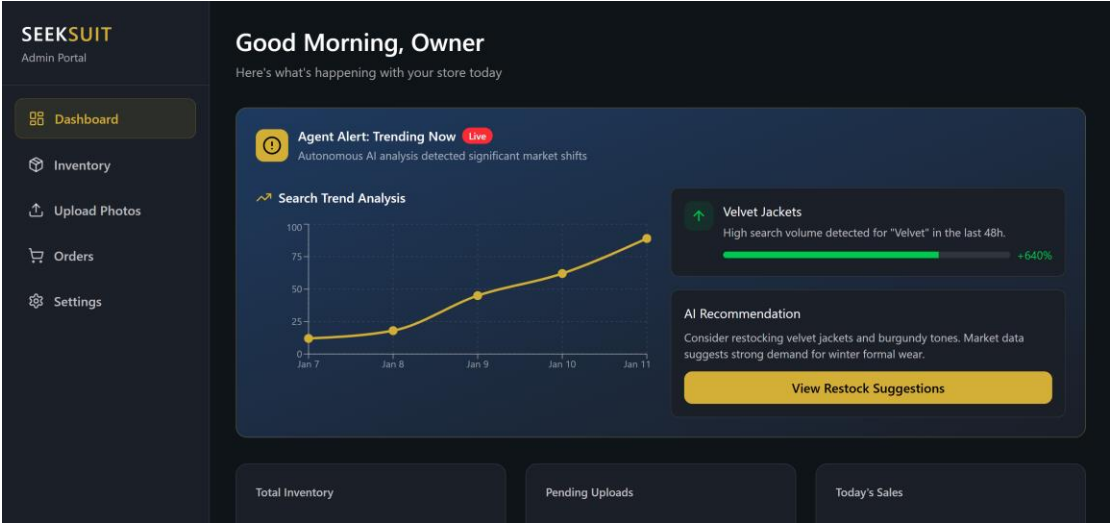
Screenshot 2: Similarity search results



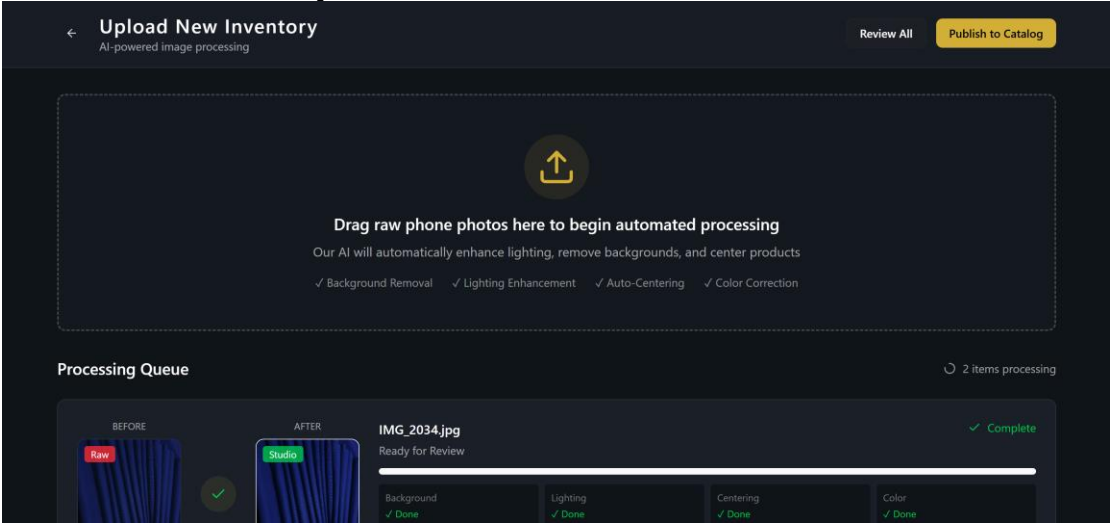
Screenshot 3: Admin login screen



Screenshot 4: Admin dashboard



Screenshot 5: Admin upload



6. Typical User Flows

1. User Flow for Customer Visual Search:

Open the website homepage → Click the 'Camera' icon in the search bar → Upload a reference photo from the device → View visually similar suits from the catalog → click on a product to view details.

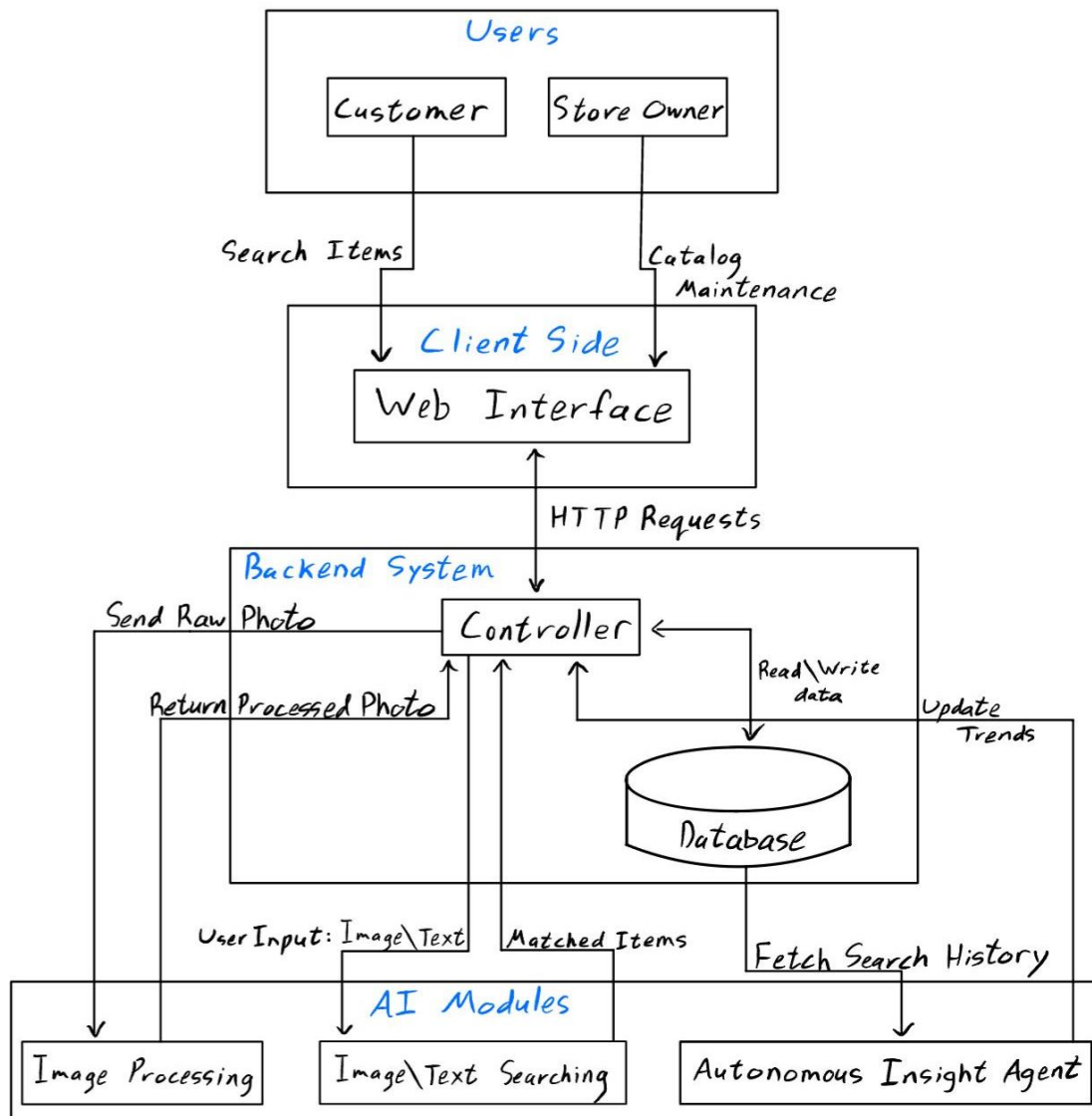
2. User Flow for Natural Language Search:

Open the website homepage → Type a descriptive query ("blue suit with two buttons") → Press 'Enter' → View results filtered by context → Refine results using standard filters → click on a product to view details.

3. User Flow for Store Owner Inventory Upload:

Open the website homepage → Click on the 'admin login' button → Log in to the Admin Dashboard → Navigate to 'Upload Inventory' → Drag and drop a raw smartphone photo → Wait for automated processing → Review the generated image and add tags → Click 'Publish' to add to the catalog.

7. Project Entity Diagram



8. Implementation

8.1 Technologies Used

Frontend: *React.js*

Backend System: *Node.js (Main Controller), Python (AI Processing)*

Database: *MongoDB*

Cloud Services: *AWS Services*

AI Services: *OpenAI API, OpenCV, Rembg, PyTorch, FAISS*